þÿ# P r o o f C h a i n J u d g e C o n c e p t G u i d e

Beginner-friendly notes for explaining every major concept used while coding ProofChain.

1. The One-Minute Project Explanation

ProofChain is a proof-of-existence system for sensitive evidence. Imagine a whistleblower has a file that proves corruption. If they upload that file to a website, the file itself can expose them. ProofChain avoids that by never uploading the file.

The browser reads the file locally and calculates a SHA-256 hash. A hash is like a digital fingerprint. The backend stores only that fingerprint, a timestamp, and a certificate ID. Later, anyone can verify the same file by hashing it again and checking whether the fingerprint exists in the registry.

The core idea:

- The file stays on the user's device.
- Only the hash goes to the server.
- The server records when that hash was first seen.
- A certificate proves the file existed at that time.

2. Proof of Existence

Proof of existence means proving that something existed at a specific time without necessarily revealing the thing itself.

In this project, the thing is a file. The proof is a record containing the file's SHA-256 hash, a server timestamp, a certificate ID, an HMAC signature, and a public verification link.

Judge answer: We prove existence by storing a timestamped hash. If the same file is presented later, it produces the same hash, proving it existed when the hash was registered.

3. Hashing

A hash is a fixed-length fingerprint created from data. A file can be 1 KB or 5 GB, but SHA-256 always outputs a 64-character hexadecimal string.

Important properties:

- Same input gives the same hash.
- A tiny change gives a totally different hash.
- You cannot rebuild the original file from the hash.
- It is extremely hard to find two different files with the same hash.

Beginner analogy: A hash is like a fingerprint for a file. You can use it to identify the file, but you cannot recreate the person from a fingerprint.

4. SHA-256

SHA-256 is a cryptographic hash function. SHA stands for Secure Hash Algorithm. 256 means the output has 256 bits. In hexadecimal form, that becomes 64 characters.

ProofChain uses SHA-256 because it is widely trusted, fast, and collision-resistant. Collision-resistant means it is practically impossible to find two different files with the same hash.

Judge answer: SHA-256 is not encryption. It is fingerprinting. Encryption is meant to be reversed with a key. Hashing is one-way.

5. Why the File Never Touches the Server

The frontend uses the browser's Web Crypto API:

```
window.crypto.subtle.digest("SHA-256", buffer)
```

This runs inside the user's browser. The code reads the file as an ArrayBuffer, computes the hash locally, and sends only the hash to the backend.

The backend receives hash, filename, filesize, and optional description. It does not receive file bytes, file contents, or an uploaded document.

Judge answer: This is not just a privacy promise. It is an architecture choice. There is no file upload endpoint in the backend.

6. Frontend Concepts

The frontend is built with React and Vite. React makes it easy to build reusable UI components. Vite provides a fast dev server and production build.

Important frontend files:

- FileHasher.jsx: lets the user select a file and calculate its hash.
- CertificateCard.jsx: shows the certificate after registration.
- VerifyPanel.jsx: verifies by certificate ID, file, or pasted hash.
- BulkVerify.jsx: verifies multiple files at once.
- TamperDemo.jsx: shows how one character change changes the hash.
- hashFile.js: contains the browser SHA-256 hashing logic.
- generatePDF.js: creates the downloadable certificate PDF.
- generateQR.js: creates the QR code for verification.

Judge answer: The frontend is responsible for privacy-sensitive hashing. The backend only receives the finished hash.

7. The Hashing Flow

1. User selects a file.
2. Browser reads the file using FileReader.
3. FileReader turns the file into an ArrayBuffer.
4. Web Crypto calculates SHA-256.
5. The hash is converted into a readable hex string.
6. Only the hash is sent to the backend.

An ArrayBuffer is a raw binary representation of the file. Cryptographic functions work on bytes, not normal text strings. Hex makes those bytes readable by turning each byte into characters from 0-9 and a-f.

8. Backend Concepts

The backend is built with Node.js and Express. Node.js lets JavaScript run on the server. Express makes it easy to create API routes.

Important backend files:

- server.js: starts the API server and connects all routes.
- schema.sql: defines the database tables.

- database.js: opens SQLite and runs the schema.
- register.js: registers new proofs.
- verify.js: verifies by certificate ID.
- verifyHash.js: verifies by hash.
- bulk.js: verifies many hashes.
- raw.js: returns plain text proof data for curl users.
- chain.js: reads proof history through parent certificates.
- verifyHMAC.js: checks certificate signatures.
- hashIP.js: hashes IP addresses before storing them.
- rateLimit.js: limits abuse.

Judge answer: The backend is intentionally hash-only. It stores proof records, not evidence.

9. Database Concepts

The project uses SQLite. SQLite is a database stored as a file. It is good for hackathons because it needs no separate database server.

The proofs table stores registered evidence fingerprints. Important columns include id, hash, filename, filesize, description, registered_at, ip_hash, expires_at, parent_cert_id, and hmac_signature.

The verifications table stores every verification attempt. Important columns include cert_id, hash_checked, result, and checked_at.

Judge answer: The database is an audit trail. Judges can inspect it and see real proof and verification records.

10. Idempotent Registration

Idempotent means doing the same action again gives the same result instead of creating duplicates. If the same file hash is registered twice, ProofChain returns the original certificate instead of creating a new timestamp.

This prevents backdating tricks. If a file was registered on Monday and submitted again on Friday, the system returns Monday's certificate.

Judge answer: Idempotency protects the timeline. Re-registering cannot create a newer or misleading proof for the same file.

11. Server-Side Timestamp

The timestamp is created by the backend with new Date().toISOString(). The client does not send the timestamp.

This matters because client timestamps can be faked. If clients could send timestamps, a malicious user could claim a file existed earlier than it really did.

Judge answer: The timestamp comes from the server, so the user cannot backdate their own proof.

12. Certificate ID

Each proof gets a certificate ID. The certificate ID lets users verify a proof without typing the full hash. The ID is generated randomly with a UUID.

Judge answer: The certificate ID is a user-friendly handle for the database proof record.

13. HMAC Signature

HMAC stands for Hash-based Message Authentication Code. It is a signature created using certificate ID, hash, timestamp, and a server secret.

The server secret is stored in an environment variable called HMAC_SECRET. If someone edits the certificate data, the HMAC check fails.

Judge answer: The HMAC makes the certificate tamper-evident. If the ID, hash, or timestamp changes, the signature no longer matches.

14. QR Code

The QR code stores the certificate verification URL. When someone scans it, they open /verify/CERTIFICATE_ID.

This is useful for printed certificates. A judge can scan the PDF and verify the proof in the live app.

Judge answer: The QR code connects a physical certificate back to the online verification record.

15. PDF Certificate

The PDF certificate is generated on the frontend with jsPDF. It includes certificate ID, filename, file size, SHA-256 hash, registration time, verification URL, and QR code.

The certificate does not include the original file.

Judge answer: The PDF is portable proof. It summarizes the registry entry and gives a verification link.

16. Verification Modes

ProofChain supports three verification modes:

- By certificate ID: the backend checks whether that ID exists.
- By file: the browser hashes the file locally and the backend checks the hash.
- By pasted hash: the backend checks a directly pasted SHA-256 hash.

Judge answer: Different users have different evidence. A certificate holder may use an ID. A file holder may verify the file directly.

17. Tamper Detection

Tamper detection works because hashes are extremely sensitive. If one letter changes, the hash changes completely.

ProofChain does not need to understand the file content. It only compares fingerprints.

Judge answer: Tamper detection is mathematical, not rule-based. The system does not guess whether a file changed. The hash proves it.

18. Bulk Verification

Bulk verification lets users check multiple files at once. The browser hashes each file locally, then sends a list of hashes to the backend.

The backend returns REGISTERED, UNREGISTERED, or INVALID_HASH.

Judge answer: Bulk verification is useful for journalists or lawyers who need to check many evidence files quickly.

19. Raw API Endpoint

The raw endpoint returns plain text instead of JSON. It returns certificate ID, hash, and timestamp.

Technical users can verify proofs with curl or terminal tools.

Judge answer: The raw endpoint makes the system usable outside the browser, especially for journalists and technical investigators.

20. Proof Chains

A proof chain links one certificate to a previous certificate. File v2 can point to File v1, and File v3 can point to File v2.

This proves an edit history without exposing any version of the file.

Judge answer: Proof chains are like Git commits for evidence. They prove document evolution without uploading document contents.

21. Rate Limiting

Rate limiting controls how often someone can call an API. ProofChain limits registration attempts to reduce spam and hash flooding.

Judge answer: Rate limiting is abuse prevention. It protects the registry from being flooded.

22. IP Hashing

The backend may need to track abuse, but storing raw IP addresses is sensitive. ProofChain hashes the IP address before storing it.

Judge answer: We do not store raw IPs. We hash them so we can rate-limit abuse with less privacy risk.

23. Environment Variables

Environment variables store configuration outside the code. Examples include PORT, FRONTEND_URL, and HMAC_SECRET.

Secrets should not be hardcoded into source files.

Judge answer: The HMAC secret belongs in environment configuration, not directly in code.

24. CORS

CORS stands for Cross-Origin Resource Sharing. It matters when the frontend and backend run on different addresses, such as localhost:5173 and localhost:3001.

Judge answer: CORS is browser-side access control for cross-origin API calls.

25. Vite Proxy

During development, Vite can proxy /api calls to the backend. This lets frontend code call /api/register instead of a full backend URL.

Judge answer: The proxy keeps frontend API calls simple in development.

26. Why This Is Not Blockchain

ProofChain is not a blockchain. It is a centralized timestamp registry. That is simpler and faster for a hackathon.

It borrows blockchain-style ideas like hashes, tamper-evident records, chain-like parent references, and public verification.

Judge answer: We used cryptographic proof without adding blockchain complexity. The goal is practical proof-of-existence, not tokenization.

27. Security Guarantees and Limits

ProofChain protects against file tampering, false denial that a file was registered, client-side timestamp manipulation, duplicate registration confusion, and accidental evidence upload.

ProofChain does not fully protect against server compromise, someone learning that a hash was registered, malware on the user's own device, or weak operational security by the user.

Judge answer: We are clear about the threat model. ProofChain protects the file contents by never uploading them, but it is not a complete anonymity system.

28. Common Judge Questions

Q: What if someone changes the file name?
A: The file hash is based on file contents, not the name. If only the name changes but the contents stay the same, the hash stays the same.

Q: What if one byte changes?
A: The SHA-256 hash changes completely. Verification fails or shows no match.

Q: Can the server read the file?
A: No. The backend never receives the file bytes.

Q: Is hashing the same as encryption?
A: No. Encryption is reversible with a key. Hashing is one-way.

Q: Can two files have the same SHA-256 hash?
A: In theory, yes. That is called a collision. In practice, finding one for SHA-256 is considered computationally infeasible.

Q: Why not store the file too?
A: Because the project is designed for whistleblowers. Storing the file creates privacy and surveillance risk.

Q: Why use SQLite?
A: SQLite is simple, inspectable, and needs no server setup. It is ideal for a hackathon proof of concept.

Q: Why use server timestamps?
A: Because client timestamps can be faked. Server timestamps make backdating harder.

Q: Why add an HMAC?
A: To make certificate fields tamper-evident. If a certificate field changes, the HMAC no longer matches.

Q: Why does duplicate registration return the old certificate?
A: To prevent timeline confusion. The first timestamp is the important one.

Q: What makes this useful in the real world?
A: Journalists, activists, lawyers, and corporate whistleblowers often need to prove evidence existed before they can safely reveal it.

29. Demo Script

1. Open the app.
2. Select a file.
3. Point out that the file is hashed in the browser.
4. Show the SHA-256 hash.
5. Click Register Proof.
6. Show the certificate and QR code.
7. Open the verify link.
8. Use the tamper demo.
9. Change one character.
10. Show that the hash changes and verification fails.

Short spoken version:

ProofChain lets a whistleblower prove evidence existed without uploading it. The browser creates a SHA-256 fingerprint locally. The server stores only that fingerprint and timestamp. Later, anyone can verify the file or certificate. If even one character changes, the hash changes and tampering is detected.

30. Best Final Pitch

Most surveillance projects try to block surveillance directly. ProofChain protects the people who expose abuse. It turns cryptography into a shield for whistleblowers: no upload, no file storage, just a timestamped mathematical fingerprint that anyone can verify.